

PRD - Earth Rover

24 August 2026

CONFIDENTIAL



ENABLE STARTUP CO., LTD

2ND FLOOR, QUANG DONG BUILDING,

108 NUI THANH STREET,

DA NANG CITY, VIETNAM

CONTACT@ENABLESTARTUP.COM

WWW.ENABLESTARTUP.COM

Table of Contents

1. Project Requirements Overview	4
2. System Overview	5
3. Team & Ownership	5
4. Project Phasing	5
5. Scope Boundaries	6
In Scope	6
Out of Scope (This Phase)	6
6. Guiding Constraints	6
7. Requirements Specifications	7
8. Deliverables	11
9. Verification Scope	12
Annex A: Extended Scope (Future Enhancements)	12
Changelog	12

Document Version	0.1 (Draft)
Document Creation Date	2026-08-10
Document Status	Draft
Company / Organization	Enable Startup
Project Type	Internal — Internship R&D
Project Name	Rover Platform (<i>working title — replace with final project name</i>)
Project Description	A WiFi-controlled rover with an onboard camera, driven from a Windows desktop controller and backed by a cloud system that stores and visualizes sensor data. The system spans three subsystems — embedded firmware, a desktop controller app, and a cloud service — that must interoperate over a jointly-defined communication protocol.
Expected Delivery Date	2026-09-17 (<i>Hardware/Firmware and Desktop App tracks</i>); <i>Cloud track dates to be confirmed separately</i>

1. Project Requirements Overview

The Rover Platform project delivers a remotely operable rover with three cooperating subsystems:

- **Hardware / Firmware** — an ESP32-based system that drives the rover's motors, acquires sensor and camera data, and communicates over WiFi.
- **Desktop Controller App** — a Windows application that connects to the rover, provides manual drive controls, displays the live video and sensor feed, and manages firmware updates.
- **Cloud System** — a backend and web dashboard that stores sensor data long-term and visualizes it, and that the desktop app can query for historical charts.

All three subsystems talk to each other over a shared communication contract (commands, telemetry, and data schema) that does not yet exist — defining it is part of the work, not a given. This PRD states *what* the system must do; the *how* — architecture, protocol design, and work breakdown — is intentionally left for each subsystem owner to propose before implementation begins.

2. System Overview

- WiFi Internet
- [Rover / ESP32] <-----> [Desktop Controller] <-----> [Cloud System]
- motors, sensors, connect/drive/view, storage, dashboard,
- camera + servo, OTA trigger, retrieve historical charts
- OTA receiver history from cloud
- **Rover ↔ Desktop:** real-time control and telemetry link (commands out, video/sensor data in).
- **Desktop ↔ Cloud:** the desktop app pulls historical sensor charts from the cloud.
- **Rover/Desktop ↔ Cloud:** sensor data reaches the cloud for storage and dashboard display (exact path — rover-direct or via desktop relay — is a design decision for the owners to propose and agree on).

3. Team & Ownership

Subsystem	Owner	Supervisor
Hardware / Firmware	Duc Nguyen	Duc Nguyen
Desktop Controller App	Haru Sakihara	Marcus + Duc
Cloud System	Shodai Tokura	Marcus + Duc

Haru and Shusuke work as a pair on the shared rover platform. Shodai's cloud track is supervised independently but must interoperate with the other two — the API/data contract between Desktop↔Cloud and Rover↔Cloud needs to be agreed across both supervision lines, not assumed.

4. Project Phasing

Phase	Weeks	Objective
Phase 1 — Discovery & Design	Week 1	Each owner reviews this PRD, explores their subsystem for gaps/edge cases, produces a Work Breakdown Structure, and proposes a high-level technical design.
Phase 2 — Core Implementation	Week 2	Build core, standalone functionality per subsystem (no cross-system integration required yet).
Phase 3 — Integration	Week 3	Connect the subsystems over the agreed protocol/API; end-to-end data flow working.

Phase 4 — Hardening & Demo	Week 4	First full operational demo early in the week, then bugfix/improvement, documentation, and final presentation.
----------------------------	--------	--

5. Scope Boundaries

In Scope

1. ESP32 firmware for rover movement control, sensor acquisition, and network communication.
2. ESP32-CAM integration: live video/image capture and transmission, servo-driven camera pan/tilt.
3. Over-the-air (OTA) firmware update mechanism.
4. Connection-loss failsafe (motion stop) and message-integrity checking (CRC) on the control link.
5. Windows desktop controller app: connect/drive UI, live video display, live sensor dashboard, OTA trigger.
6. Desktop app error handling: connection retries, low-bandwidth/low-FPS conditions, packet errors.
7. Cloud system: data ingestion, database storage of sensor data, and a web dashboard for live and historical data.
8. Desktop-to-cloud integration for retrieving historical sensor charts.
9. A jointly-owned communication protocol / data schema spanning firmware, desktop, and cloud.
10. User Guide and Verification Document for each subsystem.

Out of Scope (This Phase)

1. Final sensor selection — see Requirement HW-006; TBD during Week 1 design.
2. Image recognition, VR control, and autonomous operation — see **Annex A**.
3. Formal safety/regulatory certification of any kind.
4. Production-grade enclosure, manufacturing, or cost/BOM optimization.
5. Multi-rover or multi-operator support.

6. Guiding Constraints

These are fixed requirements, not design suggestions — the technical approach to satisfy them is still the owner's to propose.

1. **Failsafe is non-negotiable.** The rover must stop all motion automatically if it loses connection to the controller, regardless of what protocol is chosen.
2. **Message integrity is required.** Every command/telemetry message on the control link must be checkable for corruption (CRC or equivalent) before it is acted on.
3. **The control protocol is a joint contract.** Firmware and Desktop App owners must agree on the command set and data schema together — neither side defines it unilaterally.
4. **Cloud and Desktop present the same operational picture.** Whatever the desktop app can show live (video, sensor data), the cloud dashboard must be able to show historically, from data actually captured in the field.
5. **OTA must not brick the rover.** Firmware and app owners should agree on a minimum safety mechanism (e.g., confirm-before-flash, rollback, or equivalent) — the specific mechanism is a design decision.

7. Requirements Specifications

Requirement ID	Category	Title	Description	Acceptance Criteria
HW-001	Hardware/FW	Movement Control	The ESP32 shall drive the rover's motors (forward, backward, turn, stop) based on commands received over the network.	Commands issued from the controller produce the corresponding physical movement within an agreed latency budget.
HW-002	Hardware/FW	Sensor Acquisition	The ESP32 shall sample the onboard sensor set (see HW-006) and make readings available over the network link.	Sensor readings read from hardware match expected value ranges under bench test.
HW-003	Hardware/FW	Network Communication	The ESP32 shall implement a WiFi-based command/telemetry link to the desktop controller.	Round-trip command-and-telemetry exchange is demonstrated between rover and controller.
HW-004	Hardware/FW	OTA Firmware Update	Firmware shall support receiving and applying an over-the-air update without physical access to the device.	A new firmware build is pushed over WiFi and the rover boots successfully into the updated image.
HW-005	Hardware/FW	Camera Video Stream	The ESP32-CAM shall capture and transmit a live video/image stream to the controller app.	Live camera feed is visible end-to-end in the controller app.

HW-006	Hardware/FW	Sensor List Definition	The specific onboard sensor set (type, part, sampling rate, interface) shall be finalized and documented during Week 1 design. Currently TBD.	A finalized sensor list is documented and reviewed before Week 2 implementation begins.
HW-007	Hardware/FW	Camera Pan/Tilt Servo	A servo mechanism shall allow the camera to be panned/tilted on remote command.	Camera position responds correctly to pan/tilt commands within its mechanical range.
HW-008	Hardware/FW	Connection-Loss Failsafe	The rover shall stop all motion automatically if the network connection to the controller is lost or times out.	Disconnecting the link mid-drive brings the rover to a stop within an agreed timeout, verified by test.
HW-009	Hardware/FW	Message Integrity (CRC)	All command/telemetry messages shall carry a CRC or equivalent checksum.	A deliberately corrupted message is detected and discarded without causing unintended movement.
APP-001	Desktop App	Connect & Drive Controls	The app shall connect to the rover over WiFi and provide manual drive controls (e.g. joystick/buttons).	Operator can connect and drive the rover (forward/back/turn/stop) from the UI.
APP-002	Desktop App	Camera Control	The app shall let the operator control the camera pan/tilt servo remotely.	On-screen controls move the camera correctly and with acceptable responsiveness.
APP-003	Desktop App	Live Video Display	The app shall display the rover's live camera feed with latency acceptable for manual driving.	Live video is visible in-app; end-to-end latency is measured and documented.

APP-004	Desktop App	Live Sensor Dashboard	The app shall display live readings for every sensor in the finalized sensor list (HW-006).	All listed sensors update live in the UI and match firmware debug output.
APP-005	Desktop App	Connection Reliability	The app shall detect a dropped/degraded connection and automatically attempt to reconnect.	Simulated disconnect triggers a visible retry and status indicator; control resumes without restarting the app.
APP-006	Desktop App	Network/Stream Quality Handling	The app shall detect and surface low network quality (low FPS, high latency) and packet/data errors to the operator.	A throttled-bandwidth test produces a visible in-app warning; corrupted packets are logged and surfaced, not silently dropped.
APP-007	Desktop App	OTA Trigger & Monitoring	The app shall let the operator initiate an OTA update and monitor its progress/result.	Operator triggers an OTA update from the app and sees progress, success, or failure status.
APP-008	Desktop App	Historical Chart Retrieval	The app shall retrieve and display historical sensor data charts sourced from the cloud system.	Selecting a time range renders a chart backed by data from the cloud (see CLD-007).
CLD-001	Cloud	Sensor Data Ingestion	The cloud system shall receive and persist sensor data from the field pipeline.	Data submitted via the agreed ingestion path is received and stored with a timestamp.
CLD-002	Cloud	Database Storage	The cloud system shall store sensor data in a database suitable for querying by time range and sensor type.	Stored records are retrievable via query, matching what was ingested.

CLD-003	Cloud	Live/Recent Data Dashboard	The web dashboard shall display current/recent sensor data, mirroring what the desktop app shows live.	Dashboard reflects recently ingested data within an agreed refresh interval.
CLD-004	Cloud	Historical Visualization	The web dashboard shall visualize historical sensor data as charts/trends.	A selected historical time range renders correctly against stored data.
CLD-005	Cloud	Dashboard Error/Status Handling	The dashboard shall handle and surface loading states, data gaps, and device-connectivity status, consistent with APP-005/APP-006.	Simulated data gaps or stale data are visibly flagged rather than shown as if current.
CLD-006	Cloud	Remote Access & Control Parity	The web dashboard shall provide the same rover/camera access-and-control capability as the desktop app (APP-001/APP-002), via the web.	Operator can view/control the rover from the web dashboard using the same command path as the desktop app.
CLD-007	Cloud	History API for Desktop Integration	The cloud system shall expose an API the desktop app can call to retrieve historical sensor charts (feeds APP-008).	Desktop app successfully retrieves and renders a historical chart via this API.
INT-001	Integration	Shared Communication Protocol	Firmware and Desktop App owners shall jointly define and document the command set and data schema used over the control link.	A protocol/data-schema document exists, is agreed by both owners, and is used as the reference for interoperability testing.

INT-002	Integration	Cloud Data Contract	Desktop/Firmware and Cloud owners shall jointly define how sensor data reaches the cloud (path, format, frequency) and the API shape for history retrieval.	A data-contract document exists, is agreed across both supervision lines, and matches what CLD-001/CLD-007 and APP-008 actually implement.
INT-003	Integration	End-to-End Demo	The full system (rover → desktop → cloud, and cloud → desktop for history) shall be demonstrable end-to-end.	A single live demo shows: driving the rover, viewing live video/sensors, and pulling a historical chart from the cloud.

8. Deliverables

Deliverable	Description	Primary Owner(s)
Working Firmware	ESP32 firmware meeting HW-001 through HW-009.	
Operational Controller App	Windows app meeting APP-001 through APP-008.	Haru
Deployed & Operational Cloud System	Cloud backend + dashboard meeting CLD-001 through CLD-007.	Shodai
User Guide	Usage instructions per subsystem (rover setup, controller app, dashboard).	Each owner, own subsystem
Verification Document	Evidence that each subsystem's requirements were tested and pass (or documented gaps).	Each owner, own subsystem

9. Verification Scope

Each owner is expected to produce a lightweight verification record for their subsystem — one row per requirement ID above, with how it was tested and the result (pass/fail/pending, with justification for anything pending). No formal test-lab equipment is assumed; bench testing, local network simulation, and manual test procedures are acceptable given the internship timeframe. Integration requirements (INT-001 through INT-003) are verified jointly by the owners on both sides of the interface.

Annex A: Extended Scope (Future Enhancements)

Not part of the core 4-week deliverable. Only attempt these if the core requirements above are complete and stable, with time remaining.

- **A.1 — Image Recognition:** Object/obstacle detection (e.g. a pretrained model) overlaid on the live video feed, client-side or onboard.
- **A.2 — VR Control:** An alternative control interface using a VR headset/controller for immersive rover driving, in place of or alongside the joystick/button UI.
- **A.3 — Autonomous Operation:** Basic autonomous behavior (e.g. waypoint navigation or obstacle avoidance) that drives the rover without continuous manual input.

Changelog

Version	Date	Author	Description
0.1	2026-08-10	Marcus	Initial draft for intern review.